

Loops (for, while, do-while)

Loops let you execute a block of code repeatedly without writing it multiple times - until a condition is no longer true.

When to use:

Use a loop whenever you need to repeat an action a known or unknown no. of times - iterating over array, repeating input prompts, counting, or running until a condition changes.

How to use:

A) for loop:

best when you know the no. of iterations

Ex:

```
for(int i=1; i<=5; i++){  
    S.o.p("count:" + i);  
}
```

Structure: `for(initialization; condition; update)`

B) while loop:

best when the no. of iterations isn't known upfront
checks condition before running.

Ex:

```
int i=1;  
while(i<=5){  
    S.o.p("count:" + i);  
    i++;  
}
```

c) do-while loop
like while, but checks condition after running -
guarantees at least one execution.

```
Ex: int i=1;
```

```
do {  
    s.o.p("count:" + i);  
    i++;  
} while(i <= 5);
```

Where to use:

for - when the no. of repetitions is known (looping through array indices, printing 1-100).

while - when repetition depends on a condition that might change unpredictably (Ex: "keep asking until user enters a valid number")

do-while: when you need the action to happen at least once regardless of the condition (Ex: a menu that should show at least one time before checking "do you want to continue?").

Why:

- * for groups init/condition/update together - best for bounded, counter-driven loops.
- * while may run zero times if condition is false upfront; do-while always runs at least once.
- * loops must have a way for the condition to eventually become false - otherwise infinite loop (forgetting `i++` is the classic mistake).
- * for supports multiple/comma-separated expressions in each part - enables advanced patterns like two counters moving in opposite directions.

Real-life Example:

Scenario

Printing 100 receipts

Withdrawing cash until
balance = 0

ATM PIN: ask once, repeat
if wrong

washing machine cycle

Waiting for internet connection

Loop

for

while

do-while

for

while